ChatGPT said:

Writing

Your Life in Dots

A. Project details

- Page slug: your-life-in-dots
- Category: Software Development
- Level: Moderate
- Time to build: 27 hrs
- Card summary (max 100 characters): Map a lifetime as weekly dots, then explore history, habits, and milestones.
- Page subtitle (one sentence, max 160 characters): Turn a birth date into an animated weekly life map with interactive history, life chapters, time budgets, and personal milestones.
- Related program: Front-End Web Development (Vanilla JavaScript Track)

B. Guide content

1. Objective

Build a browser-based application that transforms a person's lifetime into a visual grid containing one dot for every week. Apply JavaScript date mathematics, high-performance HTML Canvas rendering, animation, and modular application architecture to calculate and visualize lived and upcoming weeks. Extend the experience with interactive historical events, personalized life chapters, time-allocation simulations, astronomical milestones, and social-media sharing.

2. Prerequisites

- JavaScript fundamentals: Functions, arrays, objects, ES6 modules, destructuring, event listeners, and asynchronous browser APIs.
- HTML5 and CSS3: Semantic elements, forms, CSS custom properties, Flexbox, Grid, and responsive media queries.
- Date and mathematical fundamentals: Leap years, UTC timestamps, integer arithmetic, modulo operations, rounding, and date comparisons.
- Basic graphics concepts: Two-dimensional coordinates, circles, canvas drawing, animation frames, and screen scaling.
- Development environment: VS Code, a modern browser, Python 3 for local hosting, and basic Git commands.

3. Tools you will use

- **HTML5, CSS3, and Vanilla JavaScript:** Build the application using native browser technologies and ES modules without React, Vue, Angular, or a build framework.
- **HTML Canvas 2D API:** Render thousands of weekly dots, overlays, animations, and high-resolution share images.
- **Browser APIs:** Use Web Audio, Local Storage, MatchMedia, Clipboard, Web Share, Blob, and URL Object APIs.
- **Visual Studio Code:** Organize modules, edit stylesheets, and debug JavaScript.
- **Google Chrome DevTools:** Inspect console errors, network requests, canvas behavior, responsive layouts, and application storage.
- **Python 3:** Start a local static HTTP server without installing project dependencies.
- **Git, GitHub, and Vercel:** Manage source control, publish the repository, and deploy the static application.

4. Steps

Step 1: Initialize the Static Application Structure

- Instruction:
  Start with a static application architecture rather than introducing a framework or backend. The project uses native HTML, CSS, and JavaScript modules, so there is no need for `npm install`, bundling, transpilation, or database configuration.
  1. Create a project folder named `life-in-dots`.
  2. Create the `js`, `assets`, and `tools` directories.
  3. Create `index.html` and `styles.css`.
  4. Add semantic application containers:
     - `#homeCard` for birth-date input.
     - `#appCard` for the completed application.
     - `#canvasWrapper` for the life grid.
     - `#tabsBar` for navigation.
     - `#drawerPanel` for interactive tools.
  5. Configure the browser entry point to load `js/main.js` as an ES module.
  6. Define CSS custom properties for light and dark themes.
  7. Establish the primary color tokens:
     - Background: `#F8FAFC`
     - Dark background: `#0F172A`
     - Primary emerald: `#059669`
     - Historical amber: `#D97706`
- Code (optional):

```
# Create the application directory
New-Item -ItemType Directory -Force life-in-dots
Set-Location life-in-dots

# Create project directories
New-Item -ItemType Directory -Force js, assets, tools

# Create initial application files
New-Item -ItemType File -Force index.html, styles.css

# Initialize Git
git init -b main
```

Create the HTML entry point:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport"
          content="width=device-width, initial-scale=1.0">

    <title>Your Life in Dots</title>

    <link rel="stylesheet" href="styles.css">
</head>

<body>
    <header id="appHeader"></header>

    <main id="app-content"></main>

    <footer id="appFooter"></footer>

    <script type="module" src="js/main.js"></script>
</body>
</html>
```

Define the initial theme tokens in `styles.css`:

```
:root {
    --bg: #F8FAFC;
```

```
    --surface: #FFFFFF;
    --primary: #059669;
    --history: #D97706;
    --text: #0F172A;
    --muted: #64748B;
    --dot-future: #CBD5E1;
}

[data-theme="dark"] {
    --bg: #0F172A;
    --surface: #1E293B;
    --text: #F8FAFC;
    --muted: #94A3B8;
    --dot-future: #475569;
}

* {
    box-sizing: border-box;
}

body {
    margin: 0;
    background: var(--bg);
    color: var(--text);
    font-family: system-ui, -apple-system, sans-serif;
}
```

- Screenshot (optional): your-life-in-dots-step-1.png
- Checkpoint: The project contains the `js`, `assets`, and `tools` directories, and `index.html` loads through a local HTTP server without HTML or stylesheet errors. The browser must recognize `js/main.js` as a JavaScript module.

Step 2: Implement the UTC Date Calculation Engine

- Instruction:
  Build the application's mathematical foundation inside `js/dates.js`. Do not use local-time date differences for lifetime calculations because timezones and daylight-saving changes can introduce incorrect day counts.
  Use integer UTC epoch days as the common calculation unit.
  Implement these functions:
  1. `isLeapYear(year)` to identify leap years.
  2. `daysInMonth(year, month)` to determine valid days in each month.

3. `dayNumber(year, month, day)` to convert a calendar date into an integer UTC day number.
4. `validateDate()` to reject incomplete, nonexistent, and future dates.
5. `getAge()` to calculate integer age using actual birthday boundaries.
6. `resolveLife()` to determine the appropriate lifespan grid.
7. `getGridPosition()` to calculate the current row and weekly column.
8. `countSundays()` to count upcoming Sundays without iterating over individual days.
9. `countFullMoons()` to calculate lunar-cycle milestones.

Important calculation rules:

- One week contains exactly seven calendar days.
- Every year is represented using 52 grid columns.
- February 29 must be handled correctly in leap and non-leap years.
- A day is a Sunday when its UTC epoch-day remainder is 3 modulo 7.
- The birthday comparison determines exact age; dividing total days by 365.25 is not an acceptable substitute.

- Code (optional):

```
// js/dates.js

const DAY_MS = 86_400_000;

export function dayNumber(year, month, day) {
    return Math.floor(
        Date.UTC(year, month - 1, day) / DAY_MS
    );
}

export function daysInMonth(year, month) {
    return new Date(
        Date.UTC(year, month, 0)
    ).getUTCDate();
}

export function isLeapYear(year) {
    return (
        year % 400 === 0 ||
        (year % 4 === 0 && year % 100 !== 0)
    );
}

export function isValidDate(year, month, day) {
```

```
    if (
        !Number.isInteger(year) ||
        !Number.isInteger(month) ||
        !Number.isInteger(day)
    ) {
        return false;
    }

    return (
        month >= 1 &&
        month <= 12 &&
        day >= 1 &&
        day <= daysInMonth(year, month)
    );
}

export function countSundays(startDay, endDay) {
    const length = endDay - startDay + 1;

    if (length <= 0) {
        return 0;
    }

    // UTC epoch day 0 was Thursday.
    // Sunday corresponds to remainder 3.
    const remainder = ((startDay % 7) + 7) % 7;

    const distance = (3 - remainder + 7) % 7;

    if (distance >= length) {
        return 0;
    }

    return (
        1 + Math.floor(
            (length - 1 - distance) / 7
        )
    );
}
```

For the grid calculation, implement the following relationship:

```
weekIndex = Math.min(
    51,
    Math.floor(daysSinceBirthday / 7)
);

currentRow = age;

livedDots = age * 52 + weekIndex;

totalDots = lifespanYears * 52;
```

Use the following reference values to debug your implementation:

- Date of birth: `2004-06-12`
- Reference date: `2026-10-23`
- Days lived: `8168`
- Completed weeks lived: `1166`
- Exact age: `22`
- Current week index: `19`
- Lived grid dots: `1163`
- Total 80-year grid dots: `4160`
- Upcoming Sundays: `3008`

Note that completed weeks and the grid's lived-dot index are separate calculations. Do not substitute one for the other.

- Screenshot (optional): your-life-in-dots-step-2.png
- Checkpoint: Verify that `29 February 2000` is accepted, `29 February 2001` and `31 February 2001` are rejected, and the reference profile produces exactly 8,168 days, age 22, 1,163 lived dots, and 3,008 upcoming Sundays.

Step 3: Build the Birth-Date Form and Application State

- Instruction:
  Create the input and application orchestration modules using `js/ui.js` and `js/main.js`. The home screen should collect the user's birth date and chosen lifespan horizon before initializing the life grid.
  1. Generate a Day dropdown containing values 1–31.
  2. Generate a Month dropdown containing January–December.
  3. Generate a Year dropdown covering the current year and the preceding 100 years.
  4. Add four lifespan selection buttons: 70, 80, 90, and 100 years.

5. Set 80 years as the default selection.
6. Implement the `Show my life` submission handler.
7. Validate all three date fields before calculating statistics.
8. Show specific validation messages rather than allowing invalid input to continue.
9. Store the submitted birth date in application memory and pass it to the calculation engine.
10. Trigger the reveal sequence only after successful validation.

Use a centralized application state instead of allowing different modules to maintain unrelated copies of the birth date and selected lifespan.

Recommended state structure:

```
const appState = {
    dob: null,

    lifePicked: 80,
    lifeUsed: 80,

    stats: null,

    activeTab: "overview",

    historyFilter: "all",

    chapters: [],

    timeBudget: null
};
```

The birth-date object should use a consistent structure:

```
const dob = {
    year: 2004,
    month: 6,
    day: 12
};
```

Display the following validation messages:

- Missing date component: `Please pick your full date of birth.`
- Invalid calendar date: `That date does not exist. Please check the day.`
- Future date: `That date is in the future.`

Privacy requirement: Do not save the submitted birth date in Local Storage or automatically append it to the URL. Persist only user preferences such as theme and sound settings. A URL date parameter may be used explicitly for development or demonstration pre-filling.

- Code (optional):

```
// Application state example

const selectedLife = {
    value: 80,
    options: [70, 80, 90, 100]
};

function buildDateFromInput(day, month, year) {
    return {
        day: Number(day),
        month: Number(month),
        year: Number(year)
    };
}

function validateAndContinue(dob, today) {
    if (!dob.day || !dob.month || !dob.year) {
        return {
            valid: false,
            message: "Please pick your full date of birth."
        };
    }

    if (
        !isValidDate(
            dob.year,
            dob.month,
            dob.day
        )
    ) {
        return {
            valid: false,
            message: "That date does not exist. Please check the da
        };
    }

    if (
```

```
        dayNumber(dob.year, dob.month, dob.day) >
        dayNumber(today.year, today.month, today.day)
    ) {
        return {
            valid: false,
            message: "That date is in the future."
        };
    }

    return {
        valid: true,
        dob
    };
}
```

- Screenshot (optional): your-life-in-dots-step-3.png
- Checkpoint: Submit an incomplete date, 31 February, and a future date. Each must display the appropriate validation message and must not initialize the grid. A valid date must produce an in-memory birth-date object and trigger the calculation flow.

Step 4: Render the High-DPI Weekly Canvas Grid

- Instruction:
  Create the visualization engine using `js/canvas.js`, `js/grid.js`, and `js/grid-draw.js`.
  Use the Canvas 2D API instead of creating thousands of individual HTML elements. This keeps rendering and pointer interaction manageable on mobile devices.
  1. Obtain the rendering context using `canvas.getContext("2d")`.
  2. Calculate canvas dimensions using the available CSS width and height.
  3. Multiply the physical canvas dimensions by `window.devicePixelRatio`.
  4. Apply `ctx.setTransform()` so drawing coordinates remain in CSS pixels.
  5. Generate 52 columns per lifespan year.
  6. Use the selected lifespan as the number of rows.
  7. Introduce additional vertical spacing after every ten rows to separate decades.
  8. Calculate each dot's X and Y positions mathematically.
  9. Color lived dots emerald and upcoming dots slate grey.
  10. Draw the current-week dot with an additional pulsing overlay.
  11. Implement `hitTestDot()` for mouse and touch interactions.
  12. Display a tooltip containing age, week number, and approximate month and year.

  Rendering architecture:
  - `canvas.js`: Canvas initialization and device-pixel-ratio scaling.

- `grid.js`: Grid geometry, pointer handling, hit testing, and tooltips.
- `grid-draw.js`: Drawing routines for empty grids, lived dots, and overlays.
- `layers.js`: Determines the visual appearance of dots in Overview and tool modes.

A 52-column, 80-row grid must contain exactly 4,160 dots. The same calculation should support 70-, 90-, and 100-year horizons.

- Code (optional):

```
// High-DPI canvas initialization

export function setupCanvas(canvas) {
    const rect = canvas.getBoundingClientRect();

    const width = rect.width;
    const height = rect.height;

    const dpr = window.devicePixelRatio || 1;

    canvas.width = Math.round(width * dpr);
    canvas.height = Math.round(height * dpr);

    const ctx = canvas.getContext("2d");

    // Keep drawing coordinates in CSS pixels.
    ctx.setTransform(dpr, 0, 0, dpr, 0, 0);

    return {
        ctx,
        width,
        height
    };
}
```

Implement the primary drawing loop using direct indexing:

```
const COLUMNS = 52;

for (let index = 0; index < totalDots; index++) {
    const row = Math.floor(index / COLUMNS);
    const col = index % COLUMNS;

    const x = padding + col * pitch + pitch / 2;
```

```
    // Insert an additional gap after every decade.
    const y =
        padding +
        row * pitch +
        Math.floor(row / 10) * decadeGap +
        pitch / 2;

    ctx.fillStyle =
        index < livedDots
            ? "#059669"
            : "#CBD5E1";

    ctx.beginPath();
    ctx.arc(x, y, dotRadius, 0, Math.PI * 2);
    ctx.fill();
}
```

- Screenshot (optional): your-life-in-dots-step-4.png
- Checkpoint: An 80-year selection renders exactly 4,160 dots in 52 columns. A 90-year selection renders 4,680 dots and a 100-year selection renders 5,200 dots. Verify that the current-week dot is distinguishable and tapping or hovering over a valid dot displays its correct information.

Step 5: Orchestrate the Reveal Animation and Procedural Audio

- Instruction:
  Create `js/reveal.js`, `js/camera.js`, `js/counter.js`, and `js/audio.js`.
  The reveal animation should convert the calculated data into a structured visual sequence rather than displaying the entire result immediately.
  Implement these stages:
  1. **Home transition:** Fade the input card away and display the results container.
  2. **Single-week introduction:** Zoom into one large dot and display `This is one week.`
  3. **Year introduction:** Reveal the first row and explain that 52 weeks form one year.
  4. **Lifetime introduction:** Zoom out to show the complete grid.
  5. **Lived-week animation:** Fill the lived dots progressively from the first week to the current week.
  6. **Current-week highlight:** Enlarge the current dot and animate its surrounding pulse ring.
  7. **Historical flashbacks:** Highlight up to three relevant past events in amber.
  8. **Perspective transition:** Count up the number of upcoming Sundays.
  9. **Results reveal:** Animate the statistics cards and enable application navigation.

Use `requestAnimationFrame()` for frame-based rendering and easing functions for smooth transitions. Avoid using fixed delays as the only mechanism for calculating animation progress.

Build a `Skip` button that cancels the active animation and audio cues and moves directly to the completed state.

Audio implementation:

Use the Web Audio API to synthesize sound instead of downloading external sound files.

- Intro swell: Noise through a modulated low-pass filter.
- Row tick: Short sound when a decade boundary is crossed.
- Current-week chime: Two sine tones at 660 Hz and 990 Hz.
- Flashback pop: Triangle-wave pitch glide from 520 Hz to 780 Hz.
- Perspective chord: Three simultaneous tones at 392 Hz, 494 Hz, and 587 Hz.

Initialize the `AudioContext` after a user interaction to avoid browser autoplay restrictions.

Provide an independent mute control.

Check the operating system's `prefers-reduced-motion` setting. When reduced motion is enabled, skip camera movements, progressive animation, and sound effects while displaying the final statistics and historical highlights.

- Code (optional):

```
// Animation easing

export function easeInOutCubic(t) {
    return t < 0.5
        ? 4 * t * t * t
        : 1 - Math.pow(-2 * t + 2, 3) / 2;
}

// Progressive lived-dot fill

export function animateLivedDots(
    livedDots,
    startTime,
    duration,
    render
) {
    function frame(now) {
        const progress = Math.min(
            1,
            (now - startTime) / duration
        );
```

```
        const eased = easeInOutCubic(progress);

        const visibleDots = Math.floor(
            livedDots * eased
        );

        render(visibleDots);

        if (progress < 1) {
            requestAnimationFrame(frame);
        }
    }

    requestAnimationFrame(frame);
}
```

- Screenshot (optional): your-life-in-dots-step-5.png
- Checkpoint: A valid birth date produces the complete reveal sequence, ending with the correct lived-week count, current-week marker, historical highlights, and upcoming Sundays. The Skip button must reach the same final result. With reduced motion enabled, the completed grid and statistics must appear without the cinematic sequence.

Step 6: Add History, Life Chapters, and Navigation

- Instruction:
  Implement the interactive exploration architecture using `js/tabs.js`, `js/history-data.js`, `js/history.js`, `js/chapters.js`, `js/chapters-ui.js`, and their respective stylesheets.
  Part A: Historical Events
  1. Create a historical dataset containing 31 curated events from Indian and world history.
  2. Store event dates in ISO format: `YYYY-MM-DD`.
  3. Give every event a unique identifier, prominence rank, category, display title, and description.
  4. Build five category filters: All, Cricket, Space, Tamil Nadu, and More.
  5. Separate events that occurred during the user's lifetime from those before birth.
  6. Display matching historical events in chronological order.
  7. Highlight matching grid dots in amber.
  8. When an event is selected, pulse its corresponding dot, open its tooltip, and scroll the canvas to the relevant row.
  9. Implement the three-event flashback selection based on relevance and historical ranking.

  Use this event data contract:

```
export const HISTORY_EVENTS = [
    {
        id: "wc-2011",
        date: "2011-04-02",
        rank: 1,
        cat: "cricket",
        title: "2011 World Cup",
        when: "India won the Cricket World Cup for the second time"
    }
];
```

The `id`, `date`, `rank`, `cat`, `title`, and `when` fields must be consistently available for every event.

Part B: Life Chapters

Allow users to divide their lived weeks into personalized life eras.

1. Build preset chapter buttons for School, College, First job, New city, and Custom.
2. Allow users to create a maximum of eight chapter cards.
3. Add a chapter name input with a 20-grapheme limit.
4. Provide start month/year and end month/year selectors.
5. Add a `Still going` checkbox for chapters that continue to the present.
6. Provide eight selectable chapter colors.
7. Recalculate dot ranges whenever a chapter is added, edited, reordered, or deleted.
8. Paint assigned lived dots using their chapter colors.
9. Paint unassigned lived dots in soft mint green.
10. Validate chapter boundaries before updating the grid.

A suitable chapter record structure is:

```
const chapter = {
    id: "chapter-01",
    name: "College",
    colorIndex: 1,
    startMonth: 7,
    startYear: 2021,
    endMonth: 5,
    endYear: 2025,
    stillGoing: false
};
```

Use the following business rules:

- The end date cannot precede the start date.
- A chapter cannot begin before the user's birth.
- A completed chapter cannot end in the future.
- An ongoing chapter uses today's date as its effective end.
- When chapters overlap, the chapter positioned lower in the list takes priority.

Part C: Navigation

Build the five application views: Overview, History, Chapters, Time, and Milestones. The active view should control which visual layer, information panel, and interaction handlers are displayed.

- Code (optional):

```
// Example chapter range validation

function validateChapter(chapter, birthMonthIndex, todayMonthIndex)
    const start =
        chapter.startYear * 12 + chapter.startMonth;

    const end = chapter.stillGoing
        ? todayMonthIndex
        : chapter.endYear * 12 + chapter.endMonth;

    if (start < birthMonthIndex) {
        return "Chapter cannot begin before birth.";
    }

    if (end < start) {
        return "End date cannot precede start date.";
    }

    if (end > todayMonthIndex) {
        return "Chapter cannot end in the future.";
    }

    return null;
}
```

- Screenshot (optional): your-life-in-dots-step-6.png

- Checkpoint: The History view must display all 31 events under the All filter and correctly filter Cricket, Space, Tamil Nadu, and More. Selecting an event must highlight its corresponding dot. The Chapters view must prevent more than eight chapters and reject invalid date ranges. Verify that overlapping chapters follow the lower-list priority rule.

Step 7: Model Time Budgets and Personal Milestones

- Instruction:
  Build the Time and Milestones engines using `js/timebudget.js`, `js/timebudget-ui.js`, `js/milestones.js`, and `js/milestones-ui.js`.
  Part A: Time Budget Engine
  Model how daily activity hours compound across a future period.
  Use six categories:

Category	Default daily hours	Constraint
Sleep	8	4–12
Work	8	0–16
Travel	1	0–6
Phone	3	0–12
Learning	1	0–8
Free time	3	Calculated remainder

  1. Implement three projection horizons: Next year (52 weeks), Next 10 years (520 weeks), and All weeks ahead.
  2. Use 0.5-hour slider increments.
  3. Compute free time as the remaining daily hours after the other five categories.
  4. Render the result using a proportional stacked horizontal bar.
  5. Convert each activity into an equivalent number of future weeks.
  6. Build the `Swap 1 hour of phone for learning` action.
  7. When activated, subtract one hour from Phone and add one hour to Learning, subject to slider constraints.
  8. Recalculate the projected allocation and animate the updated canvas dots.

  Important engineering decision:
  Do not round every activity independently using `Math.round()`. Doing so can cause the sum of allocated weeks to differ from the selected horizon.
  Use the Largest Remainder Method instead:
  - Convert daily hours into half-hour units.

- Calculate each category's exact allocation.
- Assign the integer portion initially.
- Rank remaining fractions from largest to smallest.
- Distribute unassigned weeks in that order.
- Preserve the original slider order when remainders tie.

- Code (optional):

```
// Largest Remainder Method
// 48 half-hour units represent one complete day.

export function calculateTimeSplit(hours, horizonWeeks) {
    const buckets = Object.entries(hours).map(
        ([id, dailyHours], order) => {

            const halfHours = Math.round(
                dailyHours * 2
            );

            const exactUnits =
                horizonWeeks * halfHours;

            return {
                id,
                order,
                weeks: Math.floor(exactUnits / 48),
                remainder: exactUnits % 48
            };
        }
    );

    const assigned = buckets.reduce(
        (total, bucket) => total + bucket.weeks,
        0
    );

    const remaining = horizonWeeks - assigned;

    const priority = [...buckets].sort(
        (a, b) =>
            b.remainder - a.remainder ||
            a.order - b.order
    );
```

```
    for (const bucket of priority.slice(0, remaining)) {
        bucket.weeks++;
    }

    return buckets.map(({ id, weeks }) => ({
        id,
        weeks
    }));
}
```

Use the following reference input:

```
const hours = {
    sleep: 8,
    work: 8,
    travel: 1,
    phone: 3,
    learn: 1,
    free: 3
};

const horizonWeeks = 520;
```

The expected allocation is:

```
[173, 173, 22, 65, 22, 65]
```

The sum must equal exactly 520 weeks.

Part B: Milestones Engine

Calculate personal milestones that standard birthday calendars do not display.

Implement ten fixed milestones and two dynamic milestones:

- 1,000 weeks old.
- 10,000 days old.
- 1 billion seconds old.
- 2,000 weeks old.
- 20,000 days old.
- 3,000 weeks old.
- 2 billion seconds old.
- 25,000 days old.

- 4,000 weeks old.
- 30,000 days old.
- 1,000th full moon.
- Next Mars birthday.

Use these astronomical constants:

```
const FULL_MOON_PERIOD = 29.530588853;
const MARS_ORBIT_DAYS = 686.98;
```

1. Calculate milestone dates from the user's birth date.
2. Determine whether each milestone is completed, happening today, upcoming, or outside the visible grid.
3. Highlight the next upcoming milestone in a Next-Up card.
4. Display milestone markers on the corresponding grid rows when applicable.
5. Generate calendar events in RFC 5545 `.ics` format.
6. On supported Android devices, offer the pre-filled Google Calendar intent.
7. On laptop and iOS devices, generate and download `nerd-birthday.ics`.

Treat astronomical calculations as estimates: the mean lunar period is an approximation, and missing birth-time information can introduce small calendar-date differences for certain milestones.

- Screenshot (optional): your-life-in-dots-step-7.png
- Checkpoint: For a 520-week horizon, the default time configuration must produce `[173, 173, 22, 65, 22, 65]`, with the allocations summing to exactly 520. Test the 1-hour swap and confirm that Phone decreases by one hour while Learning increases by one hour. The Milestones view must display all 12 tracked landmarks and successfully generate a valid `.ics` file with a milestone summary and date.

Step 8: Generate Share Assets and Polish Responsive Behavior

- Instruction:
  Build the sharing and final presentation features using `js/share.js`, `js/shareImage.js`, `js/theme.js`, and the relevant responsive stylesheets.
  Part A: Share Image Generation
  Do not take a screenshot of the browser window to create a share image. Generate a separate image using an offscreen HTML Canvas.
  1. Create an offscreen canvas that is not attached to the visible DOM.
  2. Draw the selected background, application title, age, lived-week statistics, and a miniature life grid.

3. Include the Wrench Wise logo and the application's web address.
4. Support two export formats:
    - Post: `1080 × 1350` pixels (4:5).
    - Status: `1080 × 1920` pixels (9:16).
5. Convert the canvas into a PNG Blob.
6. Use `navigator.share()` where the browser supports native sharing.
7. Provide a PNG download fallback named `life-in-dots.png`.

Part B: Responsive Interface

Use 900 pixels as the primary breakpoint separating mobile and laptop layouts.

Interface	Mobile	Laptop
Main layout	Single-column stack	Two-column split
Canvas	Positioned above content	Sticky left-side canvas
Navigation	Fixed bottom navigation bar	Horizontal tab control
Tool panels	Animated bottom sheet	In-place tab replacement
Dot inspection	Single tap	Mouse hover

Part C: Browser Preferences and Accessibility

1. Implement light and dark theme switching.
2. Persist the theme using the `lid-theme` Local Storage key.
3. Persist the sound preference using the `lid-muted` Local Storage key.
4. Detect the device's system theme using `prefers-color-scheme`.
5. Detect reduced-motion preferences using `prefers-reduced-motion`.
6. Ensure interactive controls can be reached using keyboard navigation.
7. Provide readable event descriptions and statistical text alongside canvas-based visual information.
8. Verify that the application's normal operation does not store or transmit the user's birth date.

- Code (optional):

```
// Theme preference

export function saveTheme(theme) {
    localStorage.setItem("lid-theme", theme);

    document.documentElement.dataset.theme = theme;
}
```

```
// Sound preference

export function saveMutePreference(isMuted) {
    localStorage.setItem(
        "lid-muted",
        isMuted ? "1" : "0"
    );
}

// Accessibility preference

export function prefersReducedMotion() {
    return window.matchMedia(
        "(prefers-reduced-motion: reduce)"
    ).matches;
}
```

- Screenshot (optional): your-life-in-dots-step-8.png
- Checkpoint: Generate both PNG formats and verify their exact pixel dimensions. Test native sharing where supported and the download fallback otherwise. At 390 × 844 pixels, the application must use its mobile navigation and bottom-sheet presentation; at 1440 × 900 pixels, it must use the two-column layout. Refresh the application and confirm that theme and mute preferences persist without persisting the birth date.

Step 9: Verify Regression Tests and Deploy the Application

- Instruction:
  Implement the final verification workflow using `js/selftest-data.js`, `js/selftest.js`, and `js/selftest-v2.js`.
  The application should not be deployed based only on whether the homepage looks correct. Its mathematical calculations, input boundaries, canvas interaction, and feature-specific behavior must be verified independently.
  Part A: Automated Testing
  Create seven automated test suites:
  1. Date mathematics: Leap years, invalid dates, age calculations, and week indexing.
  2. Sunday calculations: Calendar-day remainders and exact Sunday counts.
  3. Milestone calculations: Billion-second dates, Mars birthdays, and full-moon calculations.
  4. Time budget calculations: 24-hour conservation, week allocation, and largest-remainder behavior.
  5. History calculations: Category filtering, pre-birth events, and flashback selection.
  6. URL parameter handling: Testing, demo, lifespan, date prefill, and tab navigation.
  7. Chapters validation: Invalid boundaries, date constraints, and chapter resolution.

Connect the test harness to the `?test=1` URL parameter.

The completed project should display a structured test report containing 75 verification checks.

Part B: Manual Browser Testing

Open the following URLs:

```
http://localhost:8000/
http://localhost:8000/?test=1
http://localhost:8000/?demo=1
http://localhost:8000/?dob=2004-6-12&life=80
http://localhost:8000/?dob=2004-6-12&tab=history
http://localhost:8000/?theme=dark
```

Verify:

- The home form loads without console errors.
- The reveal sequence completes and Skip works.
- All five views open correctly.
- History filters and event selection work.
- Chapter validation and canvas recoloring work.
- Time sliders update the projected distribution.
- Milestone calendar files are generated.
- Share images have the correct dimensions.
- Mobile and desktop layouts remain usable.
- Reduced-motion and muted-audio behavior work correctly.

Part C: Run the Application Locally

Start the HTTP server from the project root:

```
python -m http.server 8000
```

Open:

```
http://localhost:8000/
```

Do not open the application directly through `file://`. Use HTTP hosting so native ES modules can load correctly.

Part D: Publish Through GitHub and Vercel

Stage and commit the verified project:

```
git add .

git commit -m "feat: build Your Life in Dots"

git remote add origin https://github.com/<your-username>/life-in-do

git push -u origin main
```

Replace the repository URL with your own GitHub repository.

Import that repository into Vercel and configure the project as a static site:

- Framework preset: Other.
- Build command: None.
- Output directory: `.` (repository root).

Vercel will serve the HTML, CSS, JavaScript, and static assets directly. Connect future deployments to your GitHub `main` branch.

Final production URL: `https://lifeindots.vercel.app`

Use your own deployed domain when publishing a separate version of the project.

- Screenshot (optional): your-life-in-dots-step-9.png
- Checkpoint: The automated test dashboard must report 75 passed checks, zero failures, and a 100% pass rate using the fixed reference date of 23 October 2026. The application must also pass the manual browser tests and load successfully from its deployed HTTPS address without build or runtime errors.

5. Expected result

- Demo video link: Pending demo recording

Your app should:

- Accept a valid birth date and lifespan selection, then generate a 52-column life grid with correctly calculated lived, current, and upcoming weeks.
- Play an approximately 16-second animated reveal with camera transitions, progressive dot filling, historical flashbacks, audio cues, and a functional Skip button.
- Provide five interactive views: Overview, History, Chapters, Time, and Milestones, including 31 historical events, customizable life chapters, and grid-based navigation.
- Calculate time allocations across six daily activity categories, display their projected weeks, and demonstrate the effect of swapping one phone hour for learning.

- Display 12 personal milestones, support calendar export, and generate downloadable or shareable 1080 × 1350 and 1080 × 1920 PNG images.

6. LinkedIn post text

I just built **Your Life in Dots**, a browser-based life visualization application, using the free project guide from @Wrench Wise.

It transforms a birth date into an animated 52-week-per-year canvas, with interactive history, personalized life chapters, time-budget simulations, and astronomical milestones.

Try it: https://lifeindots.vercel.app

#WrenchWiseProjects #JavaScript #HTML5Canvas #FrontendDevelopment

PROJECT_DETAILS(3).md